

Arquitectura Hexagonal frente a Arquitectura en Capas: un Análisis Comparativo de Puertos, Adaptadores y Testabilidad

Yeilin Moya Baltodano, Nazareth Gómez Gómez

Escuela de Informática y Computación, Universidad Nacional de Costa Rica

yeilin.moya.baltodano@est.una.ac.cr · nazareth.gomez.gomez@est.una.ac.cr

EIF509 Desarrollo de Aplicaciones Basadas en Web · II Ciclo 2026 · Prof. Elberth Adrián Garro Sánchez

Resumen—Este artículo compara la arquitectura hexagonal, o de puertos y adaptadores, con la arquitectura en capas tradicional para organizar la lógica de negocio de una aplicación web empresarial. El análisis usa fuentes formales y cinco criterios explícitos: rendimiento, complejidad, costo, mantenibilidad y seguridad, con énfasis en la testabilidad que produce la dirección de las dependencias. La arquitectura en capas favorece el arranque y cuesta menos cuando el dominio es pequeño o de vida corta. La hexagonal reduce el acoplamiento con la infraestructura y permite probar reglas de negocio sin una base de datos, a cambio de más interfaces y una curva de aprendizaje más alta. Aplicados a Savora, la plataforma de excedentes de comida del equipo sobre Spring Boot, PostgreSQL y MongoDB, esos criterios sostienen una adopción parcial. Los casos de uso dependen de puertos, y la reserva se prueba con adaptadores en memoria en una mediana de 2,0 segundos, frente a 21,7 segundos con contenedores de PostgreSQL y MongoDB. El dominio todavía usa entidades de persistencia, paginación de Spring Data y anotaciones del framework, y la prueba en memoria no verifica el rollback transaccional.

Palabras clave—arquitectura hexagonal, arquitectura en capas, puertos y adaptadores, testabilidad, acoplamiento

I. Introducción

La forma en que un equipo organiza el código de una aplicación web empresarial condiciona, mucho antes de que aparezca el primer error en producción, qué tan fácil será modificarla, probarla y hacerla crecer. Durante décadas, la arquitectura en capas —también llamada N-tier— fue la opción por defecto para separar presentación, lógica de negocio y acceso a datos [1], [2]. Sin embargo, en sistemas donde la lógica de negocio necesita evolucionar de forma independiente de la tecnología de persistencia, del framework web o de los servicios externos, ese modelo de capas apiladas tiende a filtrar detalles de infraestructura hacia el núcleo de la aplicación, dificultando las pruebas automatizadas y el reemplazo de tecnología [3].

En 2005, Alistair Cockburn formalizó una alternativa a la que llamó arquitectura hexagonal o de puertos y adaptadores, con el objetivo explícito de que una aplicación pudiera “ser accionada de igual manera por usuarios, programas, pruebas automatizadas o scripts por lotes, y ser desarrollada y probada en aislamiento de sus eventuales dispositivos de tiempo de ejecución y bases de datos” [3]. La idea central no es eliminar las capas, sino invertir la dirección de las dependencias: el núcleo de negocio define interfaces (puertos) que la infraestructura debe implementar mediante adaptadores, en lugar de que el núcleo dependa directamente de frameworks o motores de base de datos concretos.

Este artículo tiene como objetivo analizar críticamente ambos estilos arquitectónicos —capas tradicionales y hexagonal—

bajo cinco criterios técnicos explícitos, con particular atención a la testabilidad, que es el atributo más frecuentemente citado como ventaja diferenciadora de la arquitectura hexagonal en la literatura técnica y en casos de uso industriales documentados [4], [5]. El análisis no busca declarar un ganador absoluto, sino ofrecer criterios de decisión aplicables a un dominio de negocio real: Savora, la plataforma de excedente de comida que el equipo desarrolla en los laboratorios de este curso sobre Spring Boot, PostgreSQL y MongoDB.

El resto del artículo se organiza así: la sección II define ambos estilos con base en fuentes formales; la sección III realiza el análisis comparativo por criterios; la sección IV aplica esas conclusiones a Savora con el código del refactor y los tiempos medidos de sus pruebas; la sección V presenta la decisión, los límites del análisis y el trabajo futuro.

II. Marco conceptual

A. Arquitectura en capas (N-tier)

La arquitectura en capas organiza el sistema en niveles horizontales, cada uno con una responsabilidad específica, donde una capa típicamente solo puede comunicarse con la capa inmediatamente inferior (capas cerradas) [2]. Martin Fowler documentó este patrón como uno de los más comunes en aplicaciones empresariales, describiendo la separación clásica entre presentación, lógica de dominio y fuente de datos como una forma de aislar decisiones de cambio que ocurren a ritmos distintos [1]. En su forma más habitual sobre el ecosistema Java/Spring, las capas típicas son presentación (controladores REST), negocio (servicios), persistencia (repositorios) y base de datos, y la dependencia fluye en una sola dirección: de la presentación hacia la base de datos [2]. La ventaja principal de este modelo es su familiaridad: la mayoría de los equipos lo reconoce de inmediato y el tiempo de arranque de un proyecto nuevo es bajo. Su principal riesgo, señalado tanto por Fowler como por guías más recientes de arquitectura en la nube, es que las capas cerradas no impiden por sí mismas que la lógica de negocio dependa de anotaciones o tipos específicos del motor de persistencia, lo cual acopla las reglas de negocio a la tecnología elegida [1], [5].

B. Arquitectura hexagonal (puertos y adaptadores)

Cockburn definió la arquitectura hexagonal alrededor de dos ideas: (1) el núcleo de la aplicación —la lógica de dominio— no debe conocer ningún detalle de las tecnologías que lo rodean, y (2) toda comunicación entre el núcleo y el exterior ocurre a través de puertos, que son interfaces expresadas en el

lenguaje del dominio, e implementadas por adaptadores intercambiables para cada tecnología concreta [3]. Gándara distingue dos tipos de puertos según el sentido de la interacción: los puertos driver (o de entrada), que definen cómo un actor externo —un usuario, un controlador REST, una prueba— invoca un caso de uso, y los puertos driven (o de salida), que definen lo que el núcleo necesita de un actor externo, típicamente un repositorio de persistencia o un servicio de notificación [6]. La guía prescriptiva de AWS resume la diferencia central frente al modelo en capas: mientras la arquitectura en capas tiende a desplazar el foco de ingeniería hacia problemas técnicos, la arquitectura hexagonal obliga al equipo a resolver primero el problema de negocio y a tratar los mecanismos de entrada y salida como detalles secundarios y sustituibles [5]. Un ejemplo industrial documentado es el de Netflix, cuyo equipo de ingeniería de Studio adoptó puertos y adaptadores para poder sustituir, sin tocar la lógica de negocio, la fuente de datos de un servicio que pasó de leerse desde un monolito a leerse desde un servicio GraphQL, apoyándose en una batería de pruebas que verificaba primero la lógica de negocio de forma aislada mediante repositorios simulados (mocks), y solo después la integración real con cada fuente de datos [4].

III. Análisis comparativo

Para comparar ambos estilos se establecen cinco criterios técnicos explícitos: rendimiento, complejidad, costo, mantenibilidad y seguridad, resumidos en la Tabla I y desarrollados a continuación con énfasis en testabilidad, por ser el atributo con mayor evidencia empírica disponible.

Rendimiento. Ninguno de los dos estilos impone, por sí mismo, una penalización de rendimiento significativa: ambos son patrones de organización del código fuente, no arquitecturas de despliegue. La arquitectura hexagonal añade una capa adicional de indirección —la llamada a través de una interfaz de puerto en lugar de una llamada directa al repositorio concreto—, cuyo costo en tiempo de ejecución es, en la práctica, despreciable frente al costo de una consulta a base de datos o a una API externa. La diferencia de rendimiento observable entre ambos estilos suele depender más de decisiones de bajo nivel dentro de cada capa o adaptador que del estilo arquitectónico elegido.

Complejidad. Aquí la arquitectura en capas tiene una ventaja clara al inicio de un proyecto: requiere menos artefactos (menos interfaces, menos clases de mapeo) para lograr el mismo resultado funcional. La arquitectura hexagonal introduce, para cada operación relevante del dominio, al menos un puerto y uno o más adaptadores, lo que en proyectos pequeños puede sentirse como sobreingeniería [5]. Una revisión sistemática reciente sobre la adopción de arquitectura limpia (un estilo emparentado con la hexagonal) en proyectos Android reporta explícitamente estos costos iniciales: aumento de código repetitivo (boilerplate), una curva de aprendizaje más pronunciada y mayor dificultad de navegación en proyectos pequeños, costos que se magnifican en equipos con alta rotación o de vida corta [7].

Costo. El costo debe entenderse en dos horizontes: el costo de implementación inicial y el costo de mantenimiento a lo largo del ciclo de vida. La arquitectura en capas es más barata de iniciar, pero acumula lo que la literatura llama costo de

cambio: cuando la lógica de negocio depende de anotaciones del ORM o de tipos del framework web, cada cambio de tecnología obliga a tocar clases que en teoría pertenecían al dominio. La arquitectura hexagonal traslada ese costo hacia el inicio del proyecto, a cambio de reducirlo en cada cambio de tecnología posterior, tal como documentó el caso de Netflix al sustituir su fuente de datos sin modificar la lógica de negocio [4].

Mantenibilidad. Una revisión sistemática de literatura de 2019 a 2025 sobre la adopción de arquitectura limpia en el ecosistema Android, que analizó 32 estudios primarios seleccionados de 312 documentos identificados en IEEE Xplore, ACM Digital Library, Scopus y Google Scholar, encontró que la implementación literal del modelo de capas concéntricas es excepcional: solo un 23% de las aplicaciones Android analizadas mantiene una separación estricta entre dominio e infraestructura, mientras la mayoría introduce acoplamientos prácticos que facilitan el acceso a datos desde la presentación en escenarios de complejidad moderada [7]. La misma revisión documenta que la capa de dominio tiende a contaminarse con detalles tecnológicos dentro de los primeros seis meses de desarrollo intensivo cuando no existe refactorización sistemática, es decir, que la separación estricta por sí sola no garantiza mantenibilidad sin gobernanza técnica activa [7]. Esto es coherente con el riesgo de acoplamiento indirecto entre capas que Fowler ya señalaba en el modelo tradicional [1].

Seguridad. Ninguno de los dos estilos define por sí mismo la autenticación ni la autorización. En el ecosistema Spring, la documentación oficial ubica esa responsabilidad en una cadena de filtros que actúa antes del controlador, con mecanismos intercambiables de sesión o de token [8]. Tanto la arquitectura en capas como la hexagonal pueden dejar esa cadena en el borde de la aplicación. La diferencia práctica es de alcance: la hexagonal invita a tratar el filtro y el emisor de tokens como un adaptador de entrada, de modo que el dominio no reciba el framework de seguridad; la arquitectura en capas convive con el mismo filtro sin exigir esa frontera. Aislar el dominio no cifra secretos ni revoca un token. La testabilidad, en cambio, sí depende de cuán aisladas estén las reglas de negocio de sus dependencias externas [9].

Testabilidad. Guglielmo, Mariani y Denaro proponen una forma de medir la testabilidad basada en la dificultad real de generar casos de prueba efectivos mediante generación automática y análisis de mutantes, y muestran que esta dificultad no depende solo del tamaño del código sino de cuán aisladas están las unidades bajo prueba de sus dependencias externas [9]. Este hallazgo es coherente con el argumento central de la arquitectura hexagonal: al definir puertos como interfaces, el dominio puede probarse sustituyendo cada adaptador real por una implementación de prueba —un repositorio en memoria, un simulador (mock)— sin levantar una base de datos real ni un contenedor de pruebas [3], [6]. El ejemplo práctico de Ryś sobre Spring lo ilustra de forma directa: al definir el repositorio de dominio como una interfaz (OrderRepository), la prueba unitaria del servicio de dominio sustituye la implementación real por un simulador de Mockito y verifica la lógica de negocio sin ningún contexto de Spring ni conexión a base de datos [10]. La revisión de Henriquez-Miranda y Sánchez-Torres

confirma este efecto en producción: en las aplicaciones Android donde la separación de capas se respeta, incluso en configuraciones híbridas, se observan mejoras medibles en testabilidad porque la independencia del dominio respecto de

los frameworks de interfaz permite pruebas unitarias sin dependencias de infraestructura, reduciendo el tiempo de retroalimentación [7].

TABLA I
Comparación de alternativas según criterios técnicos

Criterio	Arq. en capas	Arq. hexagonal
Rendimiento	Sin penalización relevante	Indirección adicional despreciable
Complejidad	Baja al inicio; pocos artefactos	Más interfaces y adaptadores [5], [7]
Costo	Bajo al iniciar; sube si cambia la tecnología	Mayor al iniciar; baja en cambios posteriores [4]
Mantenibilidad	Riesgo de acoplamiento indirecto entre capas [1]	Cambios por adaptador; exige gobernanza [7]
Seguridad	La autenticación queda en filtros del borde [8]	El filtro puede aislarse como adaptador [8]
Testabilidad	Las pruebas suelen exigir infraestructura real	El dominio se prueba sin infraestructura [9], [10]

IV. Aplicación al dominio propio

A. El sistema y el refactor

Savora es la plataforma de excedentes de comida que el equipo construye en los laboratorios del curso. Un negocio publica un paquete sorpresa con cantidad, precio y hora límite de recogida; un cliente lo reserva; al cierre, el paquete no recogido puede donarse a una organización comunitaria. El perfil de impacto, con comidas rescatadas e insignias, se guarda en MongoDB. El resto del modelo transaccional vive en PostgreSQL. Todo corre sobre Spring Boot.

El punto de partida del laboratorio fue una arquitectura en capas: controladores REST, servicios y repositorios de Spring Data. El refactor colocó los casos de uso en el paquete domain y sustituyó la dependencia directa a esos repositorios por puertos de salida: ClientePort, ReservaPort, PaqueteSorpresaPort, DonacionPort y PerfilImpactoPort, entre otros. La infraestructura los implementa con adaptadores JPA y, para el perfil de impacto, con PerfilImpactoMongoAdapter. Los controladores REST quedan como adaptadores de entrada. Así, reservar un paquete no nombra el motor que guarda el perfil: lo pide a través del puerto. Un sistema administrativo publicado en 2026, también en Spring Boot y con un almacén relacional junto a MongoDB, documenta el mismo motivo para adoptar el patrón: separar el dominio de la persistencia y de la interfaz [11]. Savora permanece como un solo despliegue, no como microservicios.

Para que este cambio no quede solo narrado, las Fig. 1 y Fig. 2 muestran el fragmento real, tomado directamente del editor: en la versión original dependíamos de ReservaRepository, PaqueteSorpresaRepository y ClienteRepository, las interfaces concretas de Spring Data; en la versión hexagonal, el mismo archivo importa ReservaPort, PaqueteSorpresaPort y ClientePort, sin ninguna referencia a JPA.

```

1 package org.savora.domain;
2
3 import org.savora.business.dto.reservaport;
4 import org.savora.business.dto.reservaport;
5 import org.savora.business.exception.limitreservasactivasexception;
6 import org.savora.business.exception.limitreservasactivasexception;
7 import org.savora.business.exception.limitreservasactivasexception;
8 import org.savora.business.exception.limitreservasactivasexception;
9 import org.savora.data.cliente;
10
11 import org.savora.data.cliente;
12 import org.savora.data.pagatesorpresa;
13 import org.savora.data.pagatesorpresa;
14 import org.savora.data.perfilimpacto;
15 import org.savora.data.perfilimpacto;
16 import org.savora.data.perfilimpacto;
17 import org.savora.data.perfilimpacto;
18 import org.springframework.stereotype.Service;
19 import org.springframework.transaction.annotation.Transactional;
20 import org.springframework.transaction.annotation.Transactional;
21
22 import java.math.BigDecimal;
23 import java.math.BigDecimal;
24 import java.time.Clock;
25 import java.time.OffsetDateTime;
26
27 @Service
28 @Validated
29 public class ReservaService {

```

Fig. 1. Imports de ReservaService.java en la arquitectura en capas — repositorios de Spring Data (captura real del editor).

```

1 package org.savora.domain;
2
3 import org.savora.domain.dto.reservaport;
4 import org.savora.domain.dto.reservaport;
5 import org.savora.domain.exception.limitreservasactivasexception;
6 import org.savora.domain.exception.limitreservasactivasexception;
7 import org.savora.domain.exception.limitreservasactivasexception;
8 import org.savora.domain.exception.limitreservasactivasexception;
9 import org.savora.infrastructure.out.persistence.cliente;
10 import org.savora.infrastructure.out.persistence.cliente;
11 import org.savora.infrastructure.out.persistence.pagatesorpresa;
12 import org.savora.infrastructure.out.persistence.pagatesorpresa;
13 import org.savora.infrastructure.out.persistence.perfilimpacto;
14 import org.savora.infrastructure.out.persistence.perfilimpacto;
15 import org.savora.domain.cliente;
16 import org.savora.domain.pagatesorpresa;
17 import org.savora.domain.pagatesorpresa;
18 import org.savora.domain.perfilimpacto;
19 import org.savora.domain.perfilimpacto;
20 import org.savora.domain.reserva;
21 import org.savora.domain.reserva;
22 import org.savora.domain.reserva;
23 import org.savora.domain.reserva;
24 import org.savora.domain.reserva;
25 import org.savora.domain.reserva;
26 import org.savora.domain.reserva;
27 import org.savora.domain.reserva;
28 import org.savora.domain.reserva;
29 import org.savora.domain.reserva;
30 import org.savora.domain.reserva;

```

Fig. 2. Imports de ReservaService.java en la arquitectura hexagonal — puertos de dominio, sin tipos de JPA (captura real del editor).

La reserva es el punto donde las dos bases se encuentran. Antes de crearla, ReservaService pide el perfil de impacto del cliente. Ese perfil vive en MongoDB y se alcanza por PerfilImpactoPort, implementado por PerfilImpactoMongoAdapter. Con el perfil, el caso de uso calcula el máximo de reservas activas y luego cuenta, por ReservaPort, las reservas pendientes del mismo cliente en PostgreSQL. Si el conteo ya llegó al máximo, lanza LimiteReservasActivasException y no descuenta el paquete. En la arquitectura en capas, esa coordinación quedaba expresada contra los repositorios concretos de cada motor. Sigue siendo un solo caso de uso, y cada motor queda detrás de una interfaz distinta. Sustituir el documento del perfil, en la línea del reemplazo de fuente que documenta Netflix, no obliga a reescribir el conteo de reservas [4].

El cierre muestra el otro lado del puerto de salida. CierreDePaquetesService no arma la consulta de los paquetes vencidos dentro del caso de uso: le pide a PaqueteSorpresaPort los que siguen disponibles y cuya hora límite ya pasó. El propio servicio deja escrito que ese filtro, antes compuesto junto a la regla de negocio, vive ahora en el adaptador JPA. Esa es la dirección de dependencias que la sección II toma de Cockburn: el núcleo nombra lo que necesita en lenguaje del dominio y el adaptador traduce a la

tecnología [3]. El límite de esa limpieza está en el mismo método. Lo que el puerto devuelve sigue siendo la entidad PaqueteSorpresa, con sus anotaciones de persistencia, y el caso de uso la modifica y la guarda.

La parte del dominio que sí cumple el aislamiento descrito por Cockburn es el ciclo de vida del paquete. EstadoPaquete fija seis estados: disponible, reservado, recogido, donado, perdido y agotado. Solo admite las transiciones declaradas. Desde disponible el paquete puede reservarse, donarse, marcarse como perdido o agotarse. Desde reservado solo puede pasar a recogido. Una transición fuera de ese mapa lanza TransicionEstadoInvalidaException. La clase no importa Spring ni JPA. EstadoPaqueteTest comprueba, sin contenedores ni contexto de aplicación, que disponible transita a reservado, que reservado no vuelve a disponible y que un texto desconocido se rechaza. ReservaService usa esa máquina al descontar unidades: si la cantidad llega a cero el destino es agotado; si queda inventario, el destino es reservado. La regla de estados ya se prueba en aislamiento. El caso de uso que la invoca sigue armando entidades de persistencia, así que el aislamiento todavía no cubre la reserva completa.

El mismo corte parcial aparece en el cierre, que es el otro caso de uso central de Savora. CierreDePaquetesService depende de PaqueteSorpresaPort, OrganizacionPort y DonacionPort, y también recibe un Clock. No cierra un paquete cuya hora límite todavía no venció. Si hay organizaciones con capacidad de recolección suficiente, asigna el paquete a la de menor capacidad, registra la donación como aceptada y transita el paquete a donado. Si ninguna organización puede recibirlo, transita a perdido. Esas reglas viven detrás de puertos y se pueden razonar sin nombrar SQL. El servicio, igual que la reserva, está anotado como componente de Spring, corre dentro de una transacción y construye las entidades Donacion, OrganizacionComunitaria y PaqueteSorpresa. La decisión parcial cubre el flujo de publicar, reservar, cerrar y donar.

B. Decisión y evidencia medida

La decisión es adoptar la arquitectura hexagonal de forma parcial, y no cerrar un núcleo aislado del framework dentro de este laboratorio. El código actual ya muestra por qué un hexagonal estricto no está pagado. Los puertos devuelven entidades con anotaciones JPA que residen en infrastructure.out.persistence.jpa, de modo que el dominio sigue dependiendo de la infraestructura. La paginación usa Page y Pageable de Spring Data, en lugar de un tipo de página propio del dominio. Los casos de uso llevan las anotaciones Service y Transactional, así que el núcleo conoce

Spring. No hay puertos de entrada: el controlador invoca la clase concreta del caso de uso, no una interfaz driver [6]. Separar modelos de dominio y entidades, definir una paginación propia y extraer puertos de entrada sumaría mapeadores sin cambiar las reglas de reserva, de cierre ni de donación.

Donde la adopción sí se paga es en la prueba del caso de uso reservar. Ese caso rechaza una reserva pasada la hora límite, rechaza superar el máximo de reservas activas del cliente y rechaza un paquete sin cantidad. ReservaServiceTest sustituye los puertos por simuladores de Mockito y comprueba esas reglas sin contexto de Spring ni base de datos. El beneficio frente a la versión en capas es pequeño, porque un repositorio Spring Data ya era una interfaz que se podía simular [10]. La diferencia medible aparece al repetir la regla con adaptadores en memoria, en ReservaServiceEnMemoriaTest, y con Testcontainers, en ReservaServiceRollbackIT, que levanta PostgreSQL 16 y MongoDB 7.

El reloj del caso de uso se inyecta como Clock. La prueba fija el instante y no depende de la hora del servidor. Con ese reloj, ReservaServiceTest cubre el paquete inexistente, el cliente inexistente, la hora límite ya vencida, el tope de reservas activas calculado desde el perfil de impacto, la falta de unidades y el tránsito a agotado cuando se reserva la última. También comprueba el porcentaje de descuento a partir del precio original y del precio con descuento. Esas comprobaciones sustituyen los puertos por simuladores y no abren PostgreSQL ni MongoDB. Responden por la lógica. La pregunta de si la transacción deshace el cambio de cantidad, cuando el guardado de la reserva falla después de haber tocado el paquete, queda en ReservaServiceRollbackIT.

Las mediciones se hicieron el 6 de octubre de 2026, con el proyecto ya compilado y las imágenes de Docker ya presentes, tres corridas de cada clase mediante gradlew cleanTest test. La mediana del tiempo de suite JUnit fue de 2,0 s en memoria (corridas de 2,97 s, 2,01 s y 2,04 s) y de 21,7 s con contenedores (21,7 s, 18,5 s y 23,0 s). El comando Gradle, que incluye su propio arranque, medió 6,2 s y 27,2 s. Dentro de la suite en memoria, el método de negocio dura unos 0,02 s; el primer método de la clase se acerca a 2 s porque Mockito se engancha al iniciar. En la prueba con contenedores el método dura cerca de 1 s y el resto es el arranque de Spring y de las dos bases. Cuando la segunda reserva falla, el adaptador en memoria deja el paquete agotado y con cantidad cero: esa clase no observa el rollback. Comprobar que PostgreSQL deshace la actualización sigue exigiendo la prueba con contenedores.

TABLA II
Tiempo de la suite JUnit al probar la reserva, en segundos

Corrida	En memoria	Testcontainers
1	2,97	21,7
2	2,01	18,5
3	2,04	23,0
Mediana	2,0	21,7

En costo de cambio, el adaptador de Mongo ya ilustra el intercambio que documenta el caso de Netflix: sustituir el

almacén del perfil de impacto no obliga a reescribir la reserva [4]. Ese ahorro no alcanza todavía a un cambio de framework web ni de mapeador objeto-relacional, porque esas dependencias siguen dentro del dominio. El cierre de paquetes y la donación usan los mismos puertos de salida, de modo que la decisión parcial cubre el flujo principal de Savora y no solo la reserva. La autenticación con token permanece en JwtAuthFilter, fuera de los puertos, coherente con dejar la seguridad en el borde [8].

Completar un hexagonal estricto tendría un costo visible. Cada agregado que hoy viaja como entidad JPA —cliente, negocio, paquete, reserva, donación, organización y perfil de impacto— necesitaría un modelo de dominio y un mapeador dentro del adaptador. Habría que sustituir Page y Pageable por un tipo de página del propio dominio, retirar las anotaciones de Spring de los casos de uso y publicar puertos de entrada para reservar, cerrar y consultar. El equipo no midió las horas de ese trabajo. Sí puede ver que el número de clases de traducción crecería con cada agregado, sin modificar las reglas que EstadoPaquete, ReservaService y CierreDePaquetesService ya expresan. En un laboratorio de un ciclo, con un dominio que durante el curso no va a cambiar de base de datos, ese costo cae del lado que la sección III asigna al exceso de artefactos al inicio [5], [7]. La decisión deja el refactor en este punto, con la fuga documentada, y conserva las dos pruebas: la de memoria y la de contenedores.

V. Conclusiones

La comparación no produce un estilo ganador para todo dominio. La arquitectura en capas sigue siendo la de menor costo inicial cuando el dominio es pequeño y la tecnología de persistencia se mantiene. La hexagonal justifica su indirección cuando el sistema ya convive con más de un

mecanismo externo y el equipo necesita retroalimentación rápida sobre reglas de negocio. En Savora esa condición se cumple a medias. Hay dos almacenes, y la reserva se prueba sin contenedores en una mediana de 2,0 s frente a 21,7 s con PostgreSQL y MongoDB. El dominio todavía arrastra entidades JPA, tipos de Spring Data y anotaciones transaccionales. La decisión del equipo es mantener el refactor parcial durante este curso.

Los límites del análisis son concretos. Los tiempos comparan dos clases de prueba en una sola máquina, tres corridas cada una, con imágenes Docker ya descargadas; no describen el rendimiento de Savora en producción ni constituyen una muestra estadística. La evidencia de mantenibilidad citada para la arquitectura limpia proviene sobre todo del ecosistema Android [7], así que su traslado a Spring Boot es una analogía y no una medición hecha sobre este código. El caso administrativo de 2026 evalúa tiempos de respuesta de un sistema en uso, no el aislamiento de las pruebas [11]. Este estudio tampoco midió las horas ni las líneas del refactor: midió el efecto sobre una prueba. La seguridad se trató aparte de la testabilidad. El token de Savora se valida en un filtro del framework [8], y aquí no se evaluaron la revocación, el almacenamiento del secreto ni las pruebas de autorización.

El trabajo futuro es cerrar las fugas que el propio código ya deja visibles: modelos de dominio separados de las entidades, un tipo de paginación propio y puertos de entrada para los casos de uso. Con eso, la prueba en memoria podría ejercer las reglas sin construir entidades JPA. La prueba de rollback con Testcontainers debe conservarse, porque el adaptador en memoria no observa el comportamiento transaccional de PostgreSQL.

Referencias

- [1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, EE. UU.: Addison-Wesley, 2002.
- [2] H. Graça, “Layered Architecture,” @hgraca (blog), 3-ago-2017. [En línea]. Disponible: <https://herbertograca.com/2017/08/03/layered-architecture/>. [Accedido: 29-sep-2026].
- [3] A. Cockburn, “Hexagonal architecture,” alistair.cockburn.us, sep. 2005. [En línea]. Disponible: <https://alistair.cockburn.us/hexagonal-architecture/>. [Accedido: 29-sep-2026].
- [4] D. Svrtan y S. Makagon, “Ready for changes with Hexagonal Architecture,” Netflix Technology Blog, 10-mar-2020. [En línea]. Disponible: <https://netflixtechblog.com/ready-for-changes-with-hexagonal-architecture-b315ec967749>. [Accedido: 29-sep-2026].
- [5] F. Oruc, D. Goby, D. Kunce y M. Ploski, “Building hexagonal architectures on AWS,” AWS Prescriptive Guidance, Amazon Web Services, jun. 2022. [En línea]. Disponible: <https://docs.aws.amazon.com/prescriptive-guidance/latest/hexagonal-architectures/welcome.html>. [Accedido: 29-sep-2026].
- [6] C. Gándara, “Understanding Hexagonal Architecture,” DEV Community, 15-oct-2023, actualizado 18-nov-2024. [En línea]. Disponible: <https://dev.to/xoubaman/understanding-hexagonal-architecture-3gk>. [Accedido: 29-sep-2026].
- [7] C. N. Henriquez-Miranda y G. Sanchez-Torres, “Tendencias Contemporáneas en Arquitectura de Software: Evaluación de Clean Architecture en el Ecosistema Android,” *Respuestas*, vol. 31, n.º 1, pp. 75–85, ene. 2026, doi: 10.22463/0122820X.5627. Acceso abierto: <https://revistas.ufps.edu.co/index.php/respuestas/article/view/5627>.
- [8] Spring, “Spring Security Reference,” docs.spring.io, 2026. [En línea]. Disponible: <https://docs.spring.io/spring-security/reference/servlet/authentication/architecture.html>. [Accedido: 6-oct-2026].
- [9] L. Guglielmo, L. Mariani y G. Denaro, “Measuring Software Testability via Automatically Generated Test Cases,” *IEEE Access*, vol. 12, pp. 63904–63916, 2024, doi: 10.1109/ACCESS.2024.3396625. Preprint de acceso abierto: <https://arxiv.org/abs/2307.16185>.
- [10] Ł. Ryś, “Organizing Layers Using Hexagonal Architecture, DDD, and Spring,” *Baeldung*, actualizado 11-may-2024. [En línea]. Disponible: <https://www.baeldung.com/hexagonal-architecture-ddd-spring>. [Accedido: 29-sep-2026].
- [11] B. P. Tenecela Tacto, K. J. Zuñiga Iñiguez, B. Mazon-Olivo y D. A. Taylor Vivanco, “Arquitectura hexagonal y microservicios para la digitalización de procesos administrativos en una institución pública: caso de gestión de vacaciones y permisos,” *Informática y Sistemas*, vol. 10, n.º 1, pp. 79–97, 2026, doi: 10.33936/isrtic.v10i1.8441.